

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25 ¹⁰⁰

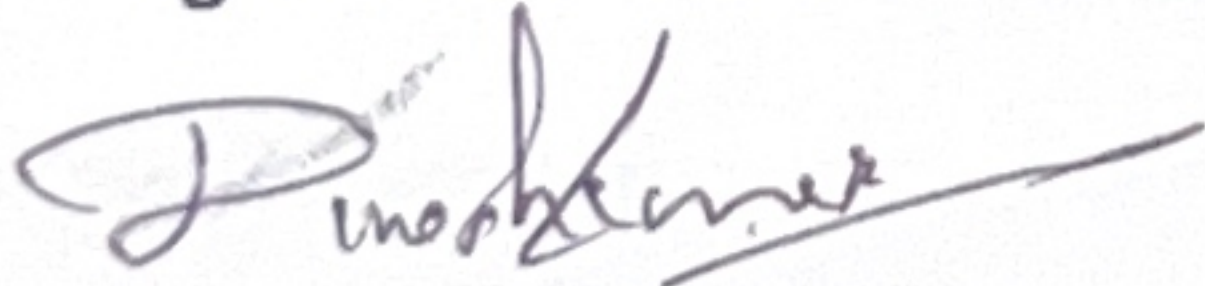
ANNEXURE A APPLICATION BASED QUESTIONS

85% 

Code of Conduct:

I DINESH KUMAR R (192312258) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



DINESH KUMAR R

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Problem Analysis

The smart city surveillance system processes continuous video feeds from multiple CCTV cameras. During peak hours, heavy data transfer between the CPU, memory, and I/O devices causes delays in threat detection and reduces overall system performance.

System Architecture & Working

The system consists of CCTV cameras, an I/O controller, system bus, CPU, memory, and an alert monitoring unit. Video data is transferred from the cameras to memory through the I/O controller. The CPU fetches, decodes, and executes instructions to analyze the video streams. After processing, alerts are generated for suspicious activities. Heavy data traffic increases memory access time and slows instruction execution.

Problem Identification

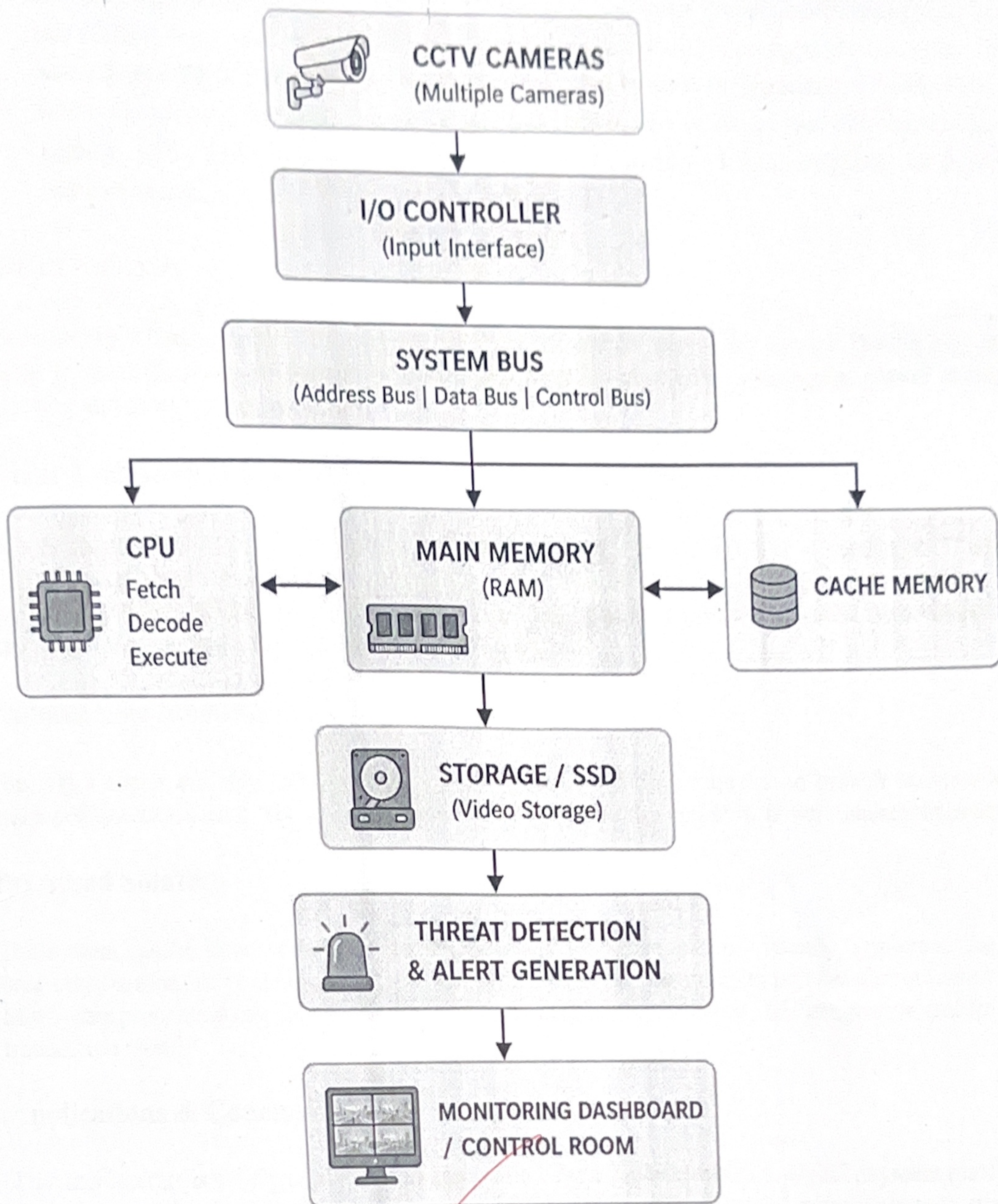
The system experiences bus congestion due to a single-bus architecture, high CPU utilization, frequent memory access, and increased CPI (Cycles Per Instruction). These factors increase execution time and reduce the efficiency of real-time surveillance.

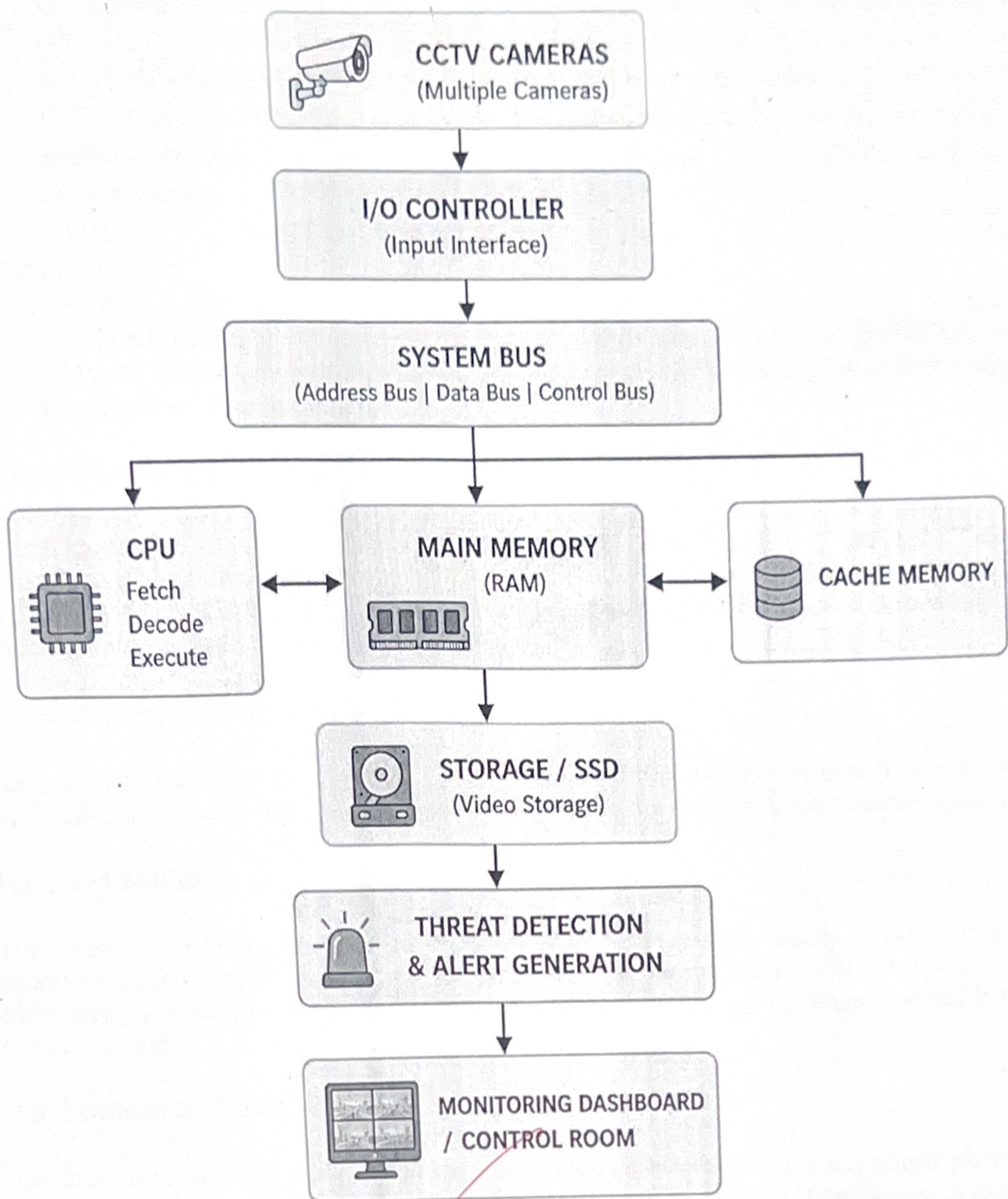
Proposed Solution

The system can be enhanced by implementing a multi-bus architecture for parallel communication between CPU, memory, and I/O devices. Cache memory should be added to reduce memory latency, DMA should handle direct data transfer between I/O devices and memory, and instruction pipelining should be used to execute multiple instructions simultaneously. These improvements reduce CPU workload and improve processing speed.

Applications & Conclusion

This architecture is suitable for smart city surveillance, airport security, traffic monitoring, industrial safety systems, and public security applications where real-time monitoring is essential. By adopting multi-bus architecture, cache memory, DMA, and pipelining, the system achieves faster execution, reduced CPI, and reliable threat detection, making it efficient for high-performance surveillance systems.





2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Problem Analysis

A real-time stock trading application processes millions of transactions every second. During high trading periods, frequent memory access and branch instructions increase CPI, resulting in slower transaction processing and delayed user responses.

System Architecture & Working

The system consists of user terminals, CPU, cache memory, main memory, and storage. Transaction requests are received from users and stored in memory. The CPU fetches, decodes, and executes trading instructions while accessing data from memory. Frequent memory access and branch instructions increase execution time, causing delays during peak trading hours.

Problem Identification

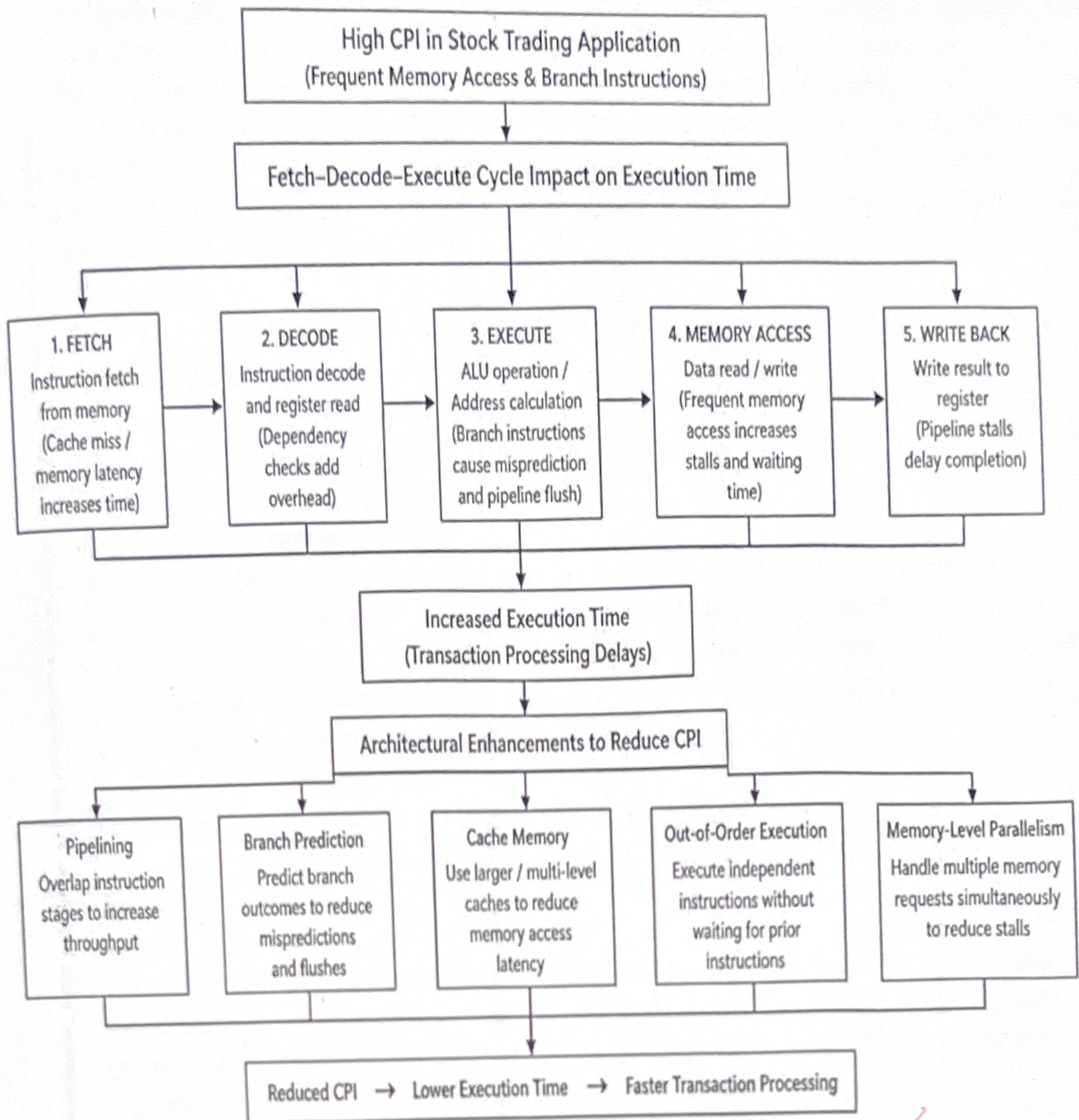
The major issues are high CPI, frequent memory access, pipeline stalls due to branch instructions, and increased execution time. These factors reduce CPU performance and slow down transaction processing.

Proposed Solution

The system can be improved by using cache memory to reduce memory latency, implementing branch prediction to minimize pipeline stalls, and adopting instruction pipelining for parallel instruction execution. Multi-core processors can also distribute workloads efficiently, reducing CPU congestion and improving transaction speed.

Applications & Conclusion

This architecture is widely used in online stock exchanges, banking systems, digital payment platforms, e-commerce applications, and financial institutions. By implementing cache memory, branch prediction, pipelining, and multi-core processing, the system achieves lower CPI, faster transaction execution, and reliable real-time financial services.



An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Problem Analysis

An industrial temperature control system uses the 8085 microprocessor to process sensor data. Improper use of addressing modes results in incorrect temperature readings, affecting the accuracy and reliability of the control system.

System Architecture & Working

The system consists of temperature sensors, an 8085 microprocessor, memory, and actuator devices. Sensors send temperature values to the microprocessor, which reads the data using different addressing modes. The CPU processes the data and sends control signals to actuators for maintaining the required temperature. Incorrect addressing causes the processor to access invalid memory locations, leading to inaccurate results.

Problem Identification

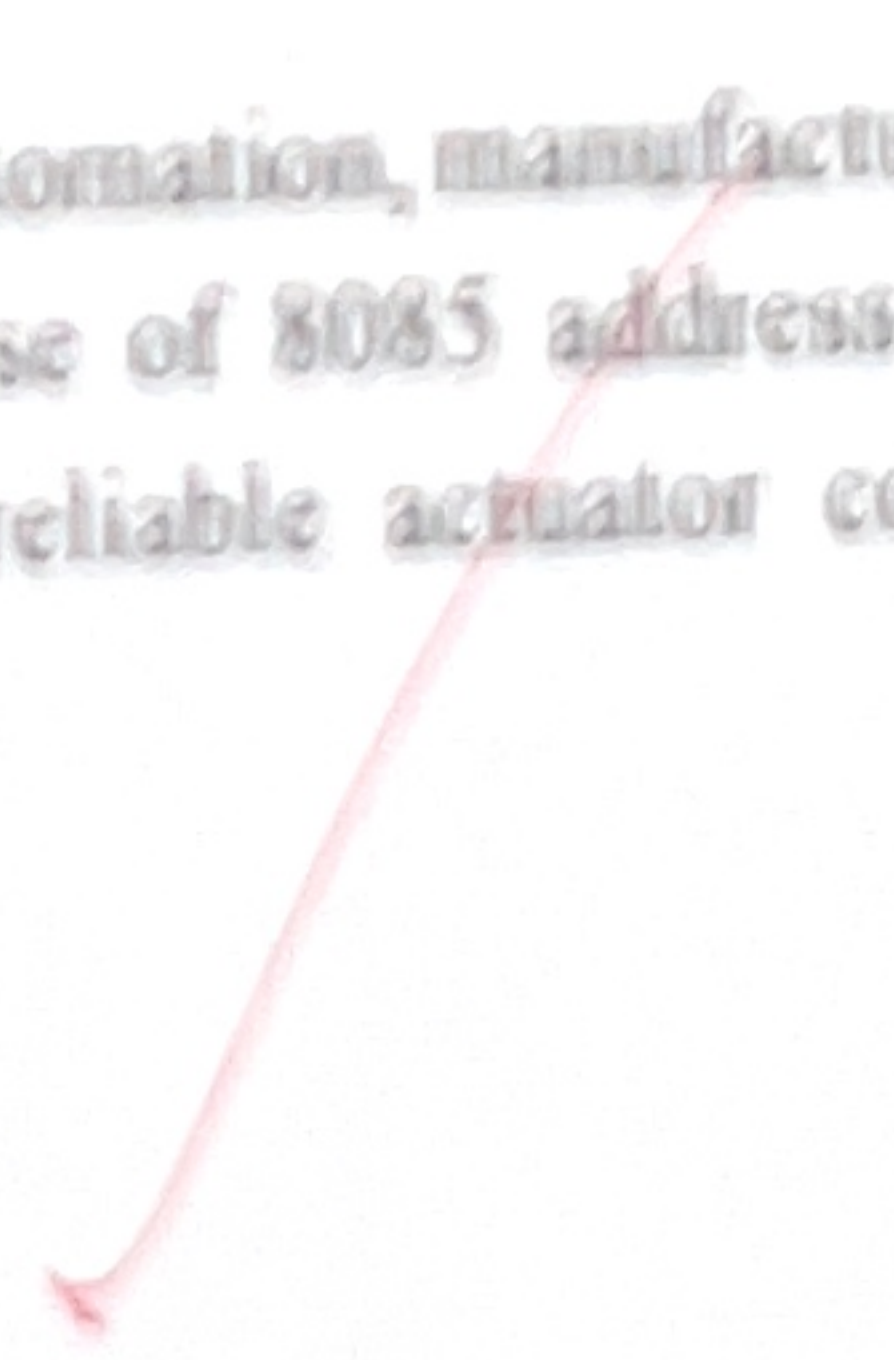
The system faces incorrect memory access, wrong sensor data processing, improper addressing mode selection, and unreliable actuator operation. These issues reduce the accuracy of industrial temperature control.

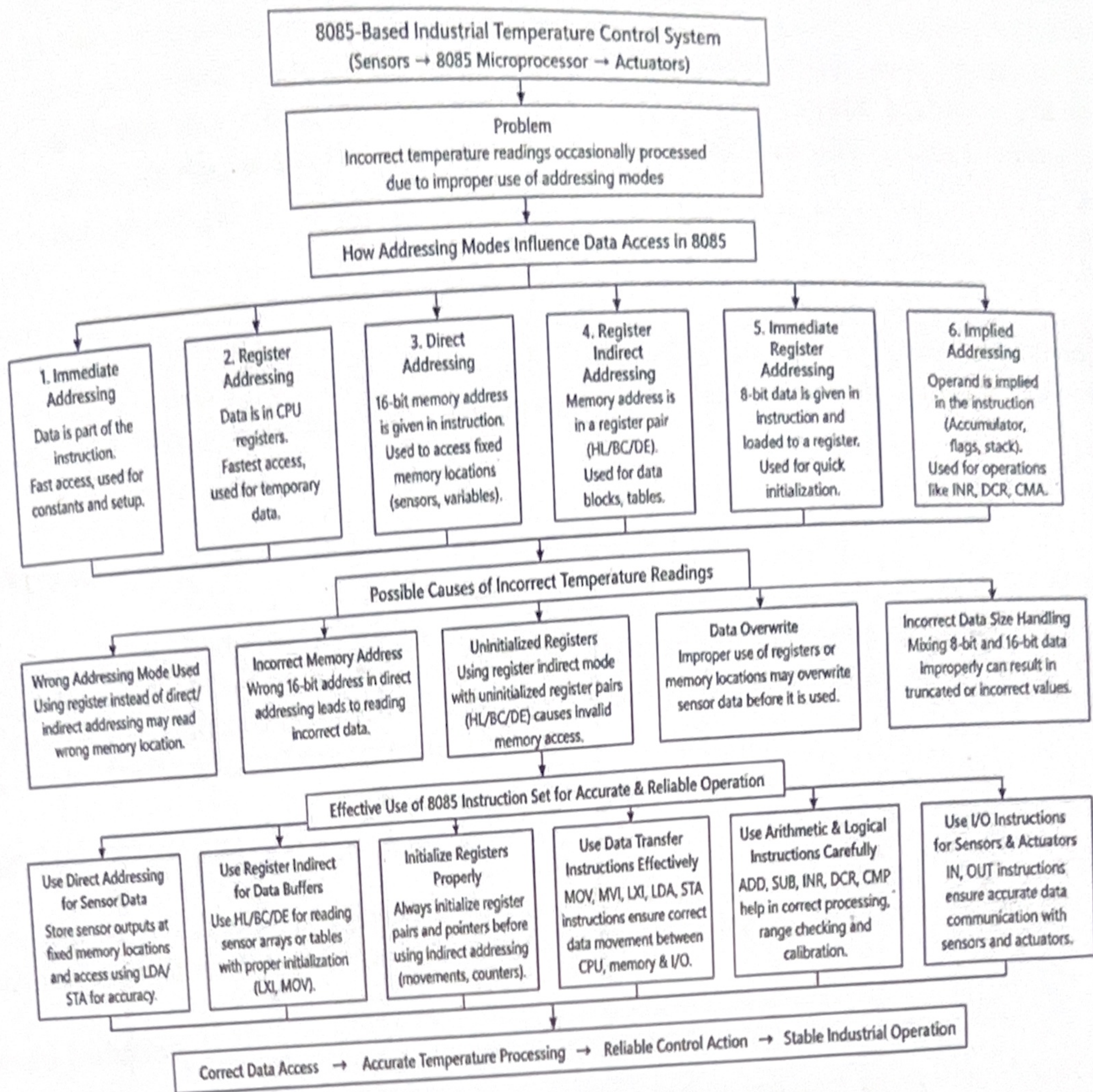
Proposed Solution

The program should use appropriate addressing modes such as Immediate, Direct, Register, and Indirect based on the application. Proper memory allocation, instruction validation, and testing of sensor inputs ensure accurate data processing. Error-checking routines can also improve system reliability.

Applications & Conclusion

This system is commonly used in industrial automation, manufacturing plants, chemical industries, power plants, and HVAC systems. Proper use of 8085 addressing modes and instruction sets ensures accurate temperature monitoring, reliable actuator control, and efficient industrial operations.





- 4 A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Problem Analysis

A computing system based on the 8086 microprocessor executes multiple programs simultaneously. Incorrect handling of memory segments leads to wrong data access, stack overflow, and program execution errors.

System Architecture & Working

The 8086 processor uses Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES) to organize memory. The CPU fetches instructions from the code segment, accesses variables from the data segment, and stores temporary data in the stack segment. Improper segment initialization causes incorrect memory references and execution failures.

Problem Identification

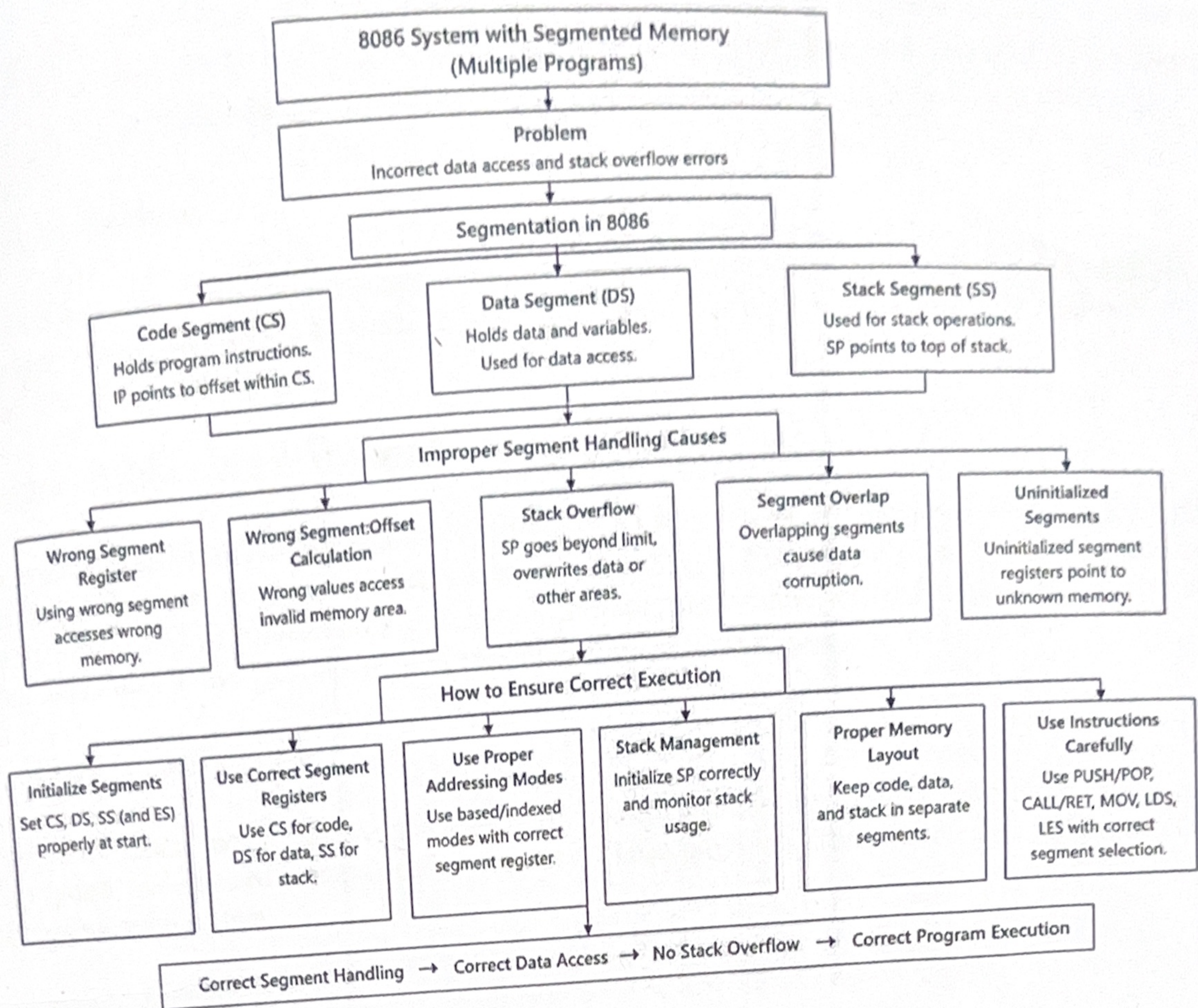
The system experiences incorrect segment register values, invalid memory access, stack overflow, and program instability. These issues reduce execution accuracy and affect multitasking performance.

Proposed Solution

The processor should initialize segment registers correctly before program execution. Adequate stack memory should be allocated, proper addressing modes should be used, and memory references should be validated to prevent stack overflow and data access errors.

Applications & Conclusion

Segmented memory architecture is used in embedded systems, industrial controllers, operating systems, and legacy computing applications. Proper segment management improves memory organization, ensures reliable multitasking, and enhances the overall efficiency of 8086-based systems.



5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Problem Analysis

A data communication system represents packet information in hexadecimal format for easy transmission. An incorrect conversion between hexadecimal and binary causes processing errors and affects data communication reliability.

System Architecture & Working

The system consists of communication devices, CPU, memory, and transmission channels. Data packets are received in hexadecimal format and converted into binary before execution. The CPU processes the binary instructions and transmits the processed information to the destination. Errors during number system conversion lead to incorrect instruction execution and data corruption.

Problem Identification

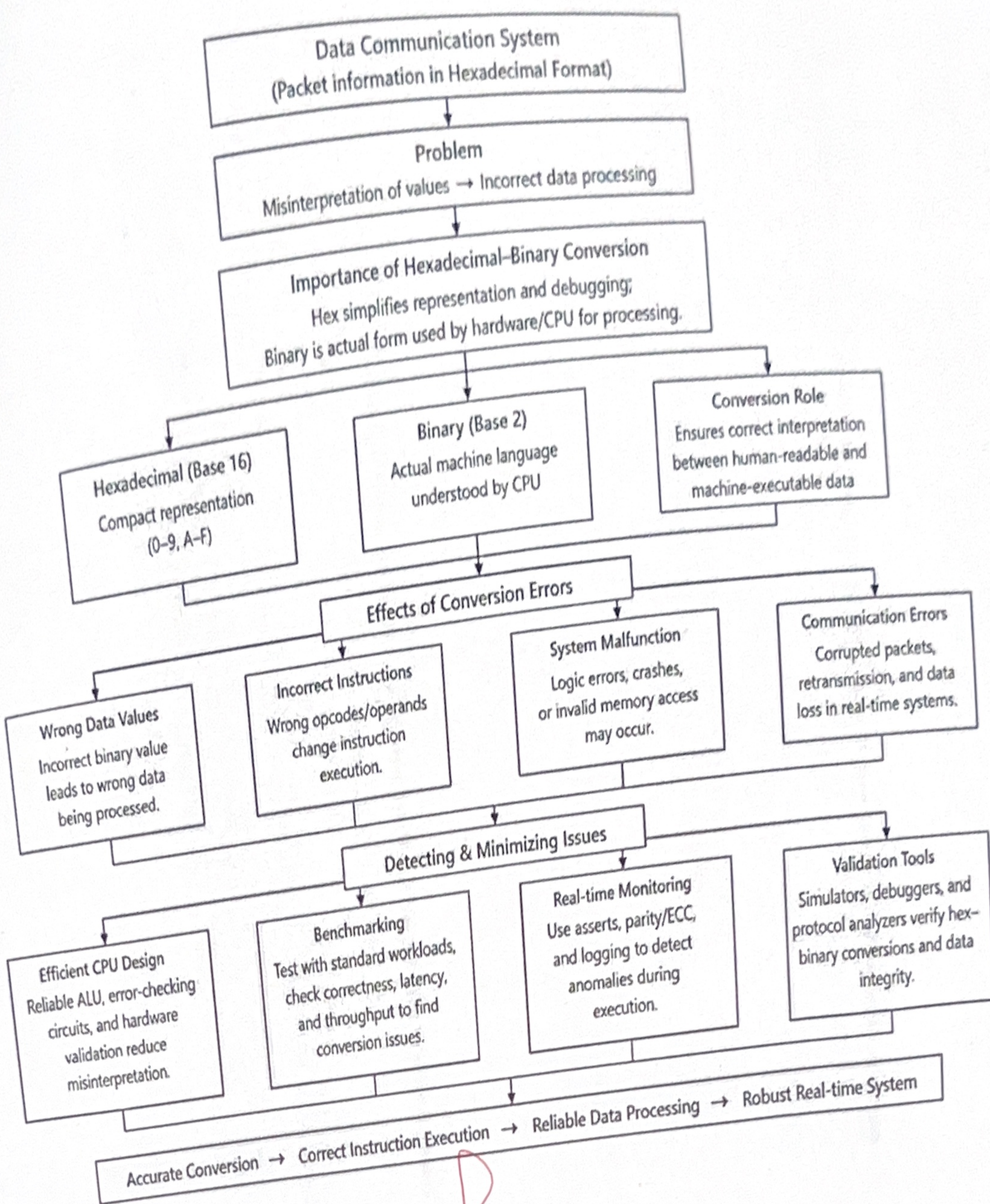
The major issues include incorrect hexadecimal-to-binary conversion, faulty instruction execution, data corruption, and reduced communication accuracy. These problems affect the overall performance of the communication system.

Proposed Solution

The system should perform accurate hexadecimal-to-binary conversion using validated algorithms. Error detection techniques such as parity checking or checksum verification should be implemented before execution. CPU benchmarking and automated validation can further improve processing accuracy and system reliability.

Applications & Conclusion

This architecture is widely used in computer networks, wireless communication, embedded systems, Internet of Things (IoT) devices, and digital communication systems. Accurate number system conversion, efficient CPU design, and benchmarking improve data integrity, reduce communication errors, and ensure reliable real-time data transmission.



MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	4	5	4	3	2	18
Q2	4	5	4	3	2	18
Q3	3	4	3	4	1.5	15.5
Q4	2	5	3	4	1.5	15.5
Q5	4	5	4	3	2	18

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁶	Applies relevant concepts correctly with minor errors ⁵	Applies basic concepts with limited accuracy ⁴	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, wellstructured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹
Clarity	10	Clear, wellorganized, and properly explained solution ²	Understandable with minor clarity issues ^{1.5}	Limited clarity and explanation ¹	Poorly presented and difficult to understand ⁰